PG-Keymaker: a Discovery Algorithm for Property Graph Keys

Gianpaolo Iuliano
University of Salerno
giaiuliano@unisa.it

Angela Bonifati
Lyon 1 University, LIRIS CNRS & IUF
angela.bonifati@univ-lyon1.fr

Andrea Mauri
Lyon 1 University & LIRIS CNRS
andrea.mauri@univ-lyon1.fr

Giuseppe Polese
University of Salerno
gpolese@unisa.it

PVLDB Reference Format:
Gianpaolo Iuliano, Angela Bonifati, Andrea Mauri, and Giuseppe Polese. PG-Keymaker: a Discovery Algorithm for Property Graph Keys. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/gianpaoloiuliano/PG-KEYMAKER.git.

1 THEORETICAL GUARANTEES

In this section, we discuss the theoretical guarantees of our approach, focusing on the main properties of *PG Keys* (i.e., soundness, completeness, and minimality), and how our restriction of the search space affects them.

Proposition 1.1 (Soundness of PG-Keymaker). *Let G be a property graph and let $p(x)$ be a scope query. Let*

$$K_{\text{out}} = \{K_1, \ldots, K_m\}$$

be the set of PG Keys returned by PG-Keymaker for $p(x)$. Then every $K_i \in K_{\text{out}}$ is a valid PG Key for $p(x)$ on G, i.e., it satisfies the IDENTIFIER constraints of Definition **??** *(EXCLUSIVE, MANDATORY, and SINGLETON).*

Proof. PG-Keymaker proceeds bottom-up on the SP-Tree.

Leaf nodes. For each leaf sub-pattern, candidate keys are obtained either (i) via UCC discovery on the set of matches, or (ii) via the external key construction based on anchor patterns. In both cases, a descriptor is retained only if it satisfies the `IDENTIFIER` constraints on the corresponding target set. Hence, all keys associated with leaf nodes are correct.

Internal nodes. For non-leaf sub-patterns, keys are constructed either by concatenating descriptors of descendant nodes or by validating candidate descriptors against the matches of the sub-pattern. By Theorem **??**, the concatenation of `IDENTIFIER` descriptors preserves `EXCLUSIVE`, `MANDATORY`, and `SINGLETON`. Additionally, the validation step ensures that only descriptors satisfying these constraints are retained.

Since every key propagated or constructed during the algorithm is either (i) directly validated or (ii) obtained from operations that preserve the `IDENTIFIER` semantic, all keys stored at each node of the SP-Tree are correct. In particular, the keys returned at the root (i.e., for the input scope $p(x)$) are valid *PG Keys*. □

As expected, Proposition 1.1 demonstrate that PG-Keymaker always return keys that are correct.

Proposition 1.2 (Conditional completeness of PG-Keymaker). *Let G be a property graph and p a scope query. Let $\mathcal{K}$ be the set of keys returned by the algorithm for p. Let $\mathcal{K}_{\text{alg}}(p)$ be the set of keys derivable for p by repeated application of the algorithm's construction rules (leaf discovery, external-key propagation via anchors, and the current upward composition/validation operator). Then the algorithm is complete w.r.t. its restricted search space:*

$$\mathcal{K} = \mathcal{K}_{\text{alg}}(p).$$

Proof. We prove, by induction on the height of a node n in the SP-Tree, that the set of keys stored by the algorithm at n equals $\mathcal{K}_{alg}(n)$.

Base case (leaves). If n is a leaf, the algorithm assigns to n exactly the set returned by find_keys (internal UCC keys, otherwise external keys via anchors). By definition of $\mathcal{K}_{alg}$, we have $\mathcal{K}_{alg}(n) = \text{Keys}(n)$, hence equality holds.

Inductive step (internal nodes). Assume the claim holds for all strict descendants of a non-leaf node n. During the upward phase, the algorithm constructs a candidate set of keys for n using: (i) its current composition operator Compose_n over the already-computed descendant-leaf key sets, (ii) keys inherited from children (if any), and/or (iii) a fallback UCC discovery on n when needed. It then filters candidates by validating them on the occurrences of n in G, keeping exactly those that satisfy the `IDENTIFIER` constraints.

By the inductive hypothesis, the sets used as inputs to Compose_n and the child-key sets are precisely $\mathcal{K}_{alg}$ of those descendants. Therefore, the set of candidates considered by the algorithm for n is exactly the set of candidates specified in the definition of $\mathcal{K}_{alg}(n)$. Since both the algorithm and $\mathcal{K}_{alg}(n)$ apply the same validation filter (`EXCLUSIVE`, `MANDATORY`, `SINGLETON`), the resulting sets coincide. Hence the claim holds for n. Applying the claim to the root node (the scope p) yields $\mathcal{K} = \mathcal{K}_{alg}(p)$. □

Proposition 1.2 shows that, although we must forgo full *completeness* due to the restriction of the search space, our approach is still able to find all keys within the restricted search space.

Proposition 1.3 (Conditional minimality). *PG-Keymaker guarantees minimality for leaf nodes. For internal nodes, minimality is ensured only when keys are produced via UCC Discovery; keys*

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

obtained through concatenation of descendant keys or propagation from children are not guaranteed to be minimal.

Proof. Leaf nodes compute keys using UCC Discovery, which returns minimal identifiers. For internal nodes, when all descendant leaves provide keys, the algorithm constructs candidates via concatenation; this operation preserves the `IDENTIFIER` constraints but does not enforce subset minimality. Similarly, when keys are propagated from children, they are validated against the `IDENTIFIER` constraints on the matches of the node, but no minimality check is performed. Minimality is enforced only in the fallback case, where UCC Discovery is executed on the matches of the internal node, returning minimal identifiers. □

Finally, minimality can be guaranteed only in specific cases, as it is the property that most impacts the efficiency of the algorithm. As Proposition 1.3 demonstrates, minimality is not ensured for keys created through concatenation using Theorem 4.1. This is expected, since verifying minimality would require checking all subsets of attributes composing the mined keys.

2 EXPERIMENTAL SCOPES.

The experimental evaluation is conducted on a set of representative scopes obtained by selecting, for each dataset, the top-8 most frequent patterns. These scopes are shown below, grouped by dataset.

	Finbench
Q1	(a:Account)-[tran:Transfer]-(a2:Account)-[wd:Withdraw]-(a3:Account)
Q2	(a:Account)-[tran:Transfer]-(a2:Account)-[tran2:Transfer]-(a3:Account)
Q3	(a:Account)-[wd:Withdraw]-(a2:Account)-[tran:Transfer]-(a3:Account)
Q4	(ln:Loan)-[dep:Deposit]-(a:Account)-[tran:Transfer]-(a2:Account)
Q5	(a:Account)-[tran:Transfer]-(a2:Account)-[rep:Repay]-(ln:Loan)
Q6	(ln:Loan)-[dep:Deposit]-(a:Account)-[wd:Withdraw]-(a2:Account)
Q7	(med:Medium)-[sig:SignIn]-(a:Account)-[wd:Withdraw]-(a2:Account)
Q8	(a:Account)-[wd:Withdraw]-(a2:Account)-[rep:Repay]-(ln:Loan)

	Synthea_100
Q1	(p:Patient)-[has:HAS_ENCOUNTER]-(en:Encounter)-[hasp:HAS_PROCEDURE]-(pro:Procedure)
Q2	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[hasp:HAS_PROCEDURE]-(pro:Procedure)
Q3	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[nx:NEXT]-(en2:Encounter)
Q4	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[hd:HAS_DRUG]-(dr:Drug)
Q5	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[hd:HAS_DRUG]-(dr:Drug)
Q6	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[he:HAS_END]-(en2:Encounter)
Q7	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[he:HAS_END]-(en3:Encounter)
Q8	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[nx2:NEXT]-(en3:Encounter)

	Synthea_200
Q1	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[nx:NEXT]-(en2:Encounter)
Q2	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[nx2:NEXT]-(en3:Encounter)
Q3	(p:Patient)-[has:HAS_ENCOUNTER]-(en:Encounter)-[hasp:HAS_PROCEDURE]-(pro:Procedure)
Q4	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[hasp:HAS_PROCEDURE]-(pro:Procedure)
Q5	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[hd:HAS_DRUG]-(dr:Drug)
Q6	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[hd:HAS_DRUG]-(dr:Drug)
Q7	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[he:HAS_END]-(en2:Encounter)
Q8	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[he:HAS_END]-(en3:Encounter)

	Synthea_300 - 1000
Q1	(p:Patient)-[has:HAS_ENCOUNTER]-(en:Encounter)-[hasp:HAS_PROCEDURE]-(pro:Procedure)
Q2	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[hasp:HAS_PROCEDURE]-(pro:Procedure)
Q3	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[nx:NEXT]-(en2:Encounter)
Q4	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[nx2:NEXT]-(en3:Encounter)
Q5	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[hd:HAS_DRUG]-(dr:Drug)
Q6	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[hd:HAS_DRUG]-(dr:Drug)
Q7	(p:Patient)-[has:HAS_ENCOUNTER]-(en1:Encounter)-[he:HAS_END]-(en2:Encounter)
Q8	(en1:Encounter)-[nx:NEXT]-(en2:Encounter)-[he:HAS_END]-(en3:Encounter)